

Ventilator Pressure Prediction — HW1 Report

Zakhar Guzii, AI301 Deep Learning

1 Data and validation

Google Brain ventilator data: each breath is a sequence of exactly 80 steps (75,450 breaths in train, 50,300 in test) with lung parameters $R \in \{5, 20, 50\}$, $C \in \{10, 20, 50\}$, time t , inspiratory valve command $u_{\text{in}} \in [0, 100]$, expiratory valve $u_{\text{out}} \in \{0, 1\}$ and target pressure p . Only rows with $u_{\text{out}} = 0$ are scored (38%); the valve opens once, at step 25–32, so inspiration is always a prefix of the breath. Main EDA findings:

- p is quantised: 950 unique values with a constant step of 0.0703 (sensor ADC). We therefore *snap* predictions to this grid (`src/postprocess.py`).
- u_{in} alone is weakly related to p (Spearman 0.16), but $\sum u_{\text{in}}$ reaches 0.81: pressure follows the inflated volume, $P = RQ + V/C + \text{PEEP}$. The correlation with u_{in} lagged by 2–4 steps (≈ 0.36) is higher than with the current value. Within a breath, future u_{in} correlates negatively with current p (down to -0.27), which is the PID controller reacting to pressure.
- 9 R_C configurations with a non-monotone effect on p ; 50_10 holds 18% of breaths, the rest 8–11%. Sampling has two regimes ($\Delta t \approx 0.032/0.034\text{s}$) whose shares depend on R_C .
- Outliers (73 breaths with negative pressure, 74 with $\max p < 3$, all $R_C = 50_10$, near-zero u_{in}) also occur in test, so we keep them.
- No train/test shift: all shares and distributions match, and adversarial validation (logistic regression on per-breath signals) gives ROC-AUC 0.499 for train vs. test and 0.491 for train vs. val.

Validation. A single 80/20 hold-out grouped by `breath_id` (rows of a breath are dependent) and stratified by R_C . K-fold was rejected: 15k validation breaths already give a stable estimate, and K-fold would multiply NN cost by K. **Metrics** (all on $u_{\text{out}} = 0$, `src/metrics.py`): MAE is the competition metric; RMSE flags rare large misses; ME shows systematic bias; R^2 compares against a constant. Reference: predicting the median gives MAE 7.52, a linear regression gives 4.97.

2 Features

`add_features` builds 31 per-step columns: raw t , u_{in} , u_{out} ; Δt ; $S_i = \sum_{j \leq i} u_j$; lags u_{i-k} and leads u_{i+k} ($k = 1..4$, zero-padded); differences $u_i - u_{i-k}$ and $u_{i+k} - u_i$; per-breath $\max u$; one-hot R_C (9 columns). Lags cover the delayed response, leads and $\max u$ expose the controller’s reaction (the whole breath is known in advance). *History.* The first version one-hot encoded R and C separately with `drop_first`; it was replaced by a 9-way R_C one-hot because the $R \times C$ effect is not additive. Rejected after EDA: the area $\sum u \Delta t$ ($\rho = 0.997$ with S , interchangeable), the distance to the maximum (a linear combination of $\max u$ and u), the deviation from the breath mean ($\rho = -0.04$), inverted u_{out} (duplicate), and rolling/EWM/expanding statistics (redundant with lags and S). The physics features Ru_{in} and $\sum u \Delta t / C$ were planned as an NN ablation but not run.

3 LightGBM

LightGBMRegressor: objective `mae`, learning rate 0.3, 1000 leaves, unlimited depth, 511 bins, at least 100 samples per leaf, $\ell_1 = 0.1$, $\ell_2 = 1.0$, 5000 trees, early stopping after 100 rounds on val MAE. Validation is scored with sample weight $\mathbb{1}[u_{\text{out}} = 0]$, so the stopping criterion matches the metric. Two variants: trained on all 4.83M rows (val MAE 0.6199) and only on inspiratory rows (1.83M, 0.6157). The second is better despite $2.6\times$ less data: expiration adds nothing to scored rows. *History.* The first runs used 30,000 trees and took hours; six runs in MLflow were interrupted (one hung overnight). Beyond ~ 3000 trees val MAE improved by < 0.002 per 500 trees, so the cap became 5000 (early stopping never fired). After RMSE was added as a second metric, early stopping had to be restricted to `first_metric_only`.

4 Data pipeline: Dataset $\rightarrow$ Collator $\rightarrow$ DataLoader

`make_loaders` runs `load_data` (sort by breath and time) $\rightarrow$ `rc_strata` $\rightarrow$ `add_features` $\rightarrow$ `split_breaths` $\rightarrow$ `compute_norm_stats` on the *train part only* (μ, σ per column; one-hot columns get $\mu = 0, \sigma = 1$, i.e. stay untouched).

- **BreathDataset** does all the work once in `__init__`. Since every breath is 80 consecutive rows, standardised features become one tensor $\mathbf{X} \in \mathbb{R}^{n \times 80 \times 31}$ by a zero-copy `view`, together with the mask $\mathbf{M} = [u_{\text{out}} = 0] \in \{0, 1\}^{n \times 80}$ and the target $\mathbf{Y} \in \mathbb{R}^{n \times 80}$ (None for test). `__getitem__(i)` only indexes: it returns the tuple $(\mathbf{X}_i, \mathbf{M}_i, \mathbf{Y}_i)$. One item is one breath, so shuffling never mixes steps of different breaths.
- **BreathDatasetCollator** receives a list of B such tuples, transposes it with `zip(*batch)` and `torch.stack` each part into a dict `{features: [B, 80, 31], mask: [B, 80], target: [B, 80]}`; `target` is added only if present, so the same code serves train, val and test.
- **DataLoader:** $B = 256$ with `shuffle=True` for train (236 steps/epoch), $B = 1024$ in order for val/test.

History. The pipeline was rewritten twice. v1 scanned the whole dataframe for each item (`df[df.breath_id==id]`); v2 used precomputed `groupby` indices with `iloc`, but the collator still converted pandas to tensors and normalised every batch (≈ 35 s per pass over train). v3 (above) moved normalisation into the Dataset and precomputes tensors: one pass takes < 2 s.

5 Neural network

5.1 Idea

Gradient boosting looks at one row at a time, so it learns about the rest of the breath only through the lags we computed by hand. Pressure, however, depends on the whole history: how much air has already entered the lung, and how the controller reacted to it. Our network therefore takes an *entire breath* as input and predicts all 80 pressures at once. Inside, it alternates two kinds of operations: one that mixes information *between time steps* (our custom layer, TimeLinear) and one that transforms the features *within each step* (a small MLP). Six such blocks are stacked with residual connections (Fig. 1).

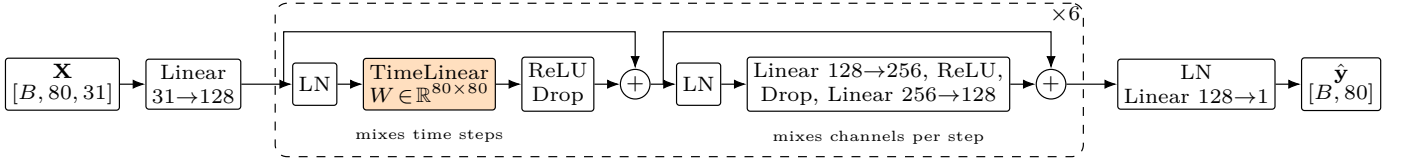


Figure 1: PressureNet (441,953 parameters): input projection, 6 pre-LN residual blocks, head.

5.2 Notation

All formulas describe a single breath; in code the same operations run on a batch of B breaths in parallel. A breath is a table with one row per time step, so every quantity below is a matrix whose **row t belongs to time step t** .

- $T = 80$ time steps, $F = 31$ input features, $d = 128$ hidden width.
- $\mathbf{X} \in \mathbb{R}^{T \times F}$: standardised input features of the breath; row t holds the 31 features of step t .
- $\mathbf{H}^{(\ell)} \in \mathbb{R}^{T \times d}$: hidden state after block ℓ ($\ell = 0$ is the input projection, $\ell = 6$ the last block). Row t is a learned 128-number description of step t . $\tilde{\mathbf{H}}^{(\ell)}$ is the intermediate state inside block ℓ , and $\mathbf{Z}$ denotes the input of a layer in general.
- LN: LayerNorm, applied to each row separately: the 128 numbers of a step are shifted to mean 0 and scaled to variance 1, then multiplied and shifted by learned vectors. Drop: dropout, which zeros 10% of the values during training.
- $\mathbf{y}, \hat{\mathbf{y}} \in \mathbb{R}^T$: true and predicted pressure; $\mathbf{M} \in \{0, 1\}^T$: mask of scored steps ($u_{\text{out}} = 0$).
- $\mathbf{1}$: a column of ones, used to add the same bias vector to every row.

5.3 TimeLinear: our custom layer

A standard `nn.Linear` works on each row separately: it takes the 128 numbers of step t and returns new numbers for the same step t . It can never see what happened at other steps. TimeLinear works the other way round, *along the time axis*. It holds one learnable matrix $W \in \mathbb{R}^{T \times T}$ and a bias $b \in \mathbb{R}^T$ and multiplies the input from the left:

$$\text{TimeLinear}(\mathbf{Z}) = W\mathbf{Z} + b\mathbf{1}^\top, \quad \text{i.e.} \quad \text{TimeLinear}(\mathbf{Z})_{t,c} = \sum_{s=1}^T W_{ts} Z_{s,c} + b_t.$$

In words: the new value of channel c at step t is a weighted sum of the *same* channel over all 80 steps, and row t of W says how much every step s contributes to step t . The same W is shared by all 128 channels and all breaths, so the layer has only $80 \cdot 80 + 80 = 6,480$ parameters.

A worked example: why the layer sees history. Take one channel of a short breath with 4 steps, for example an estimate of air flow, $\mathbf{z} = (10, 20, 30, 40)^\top$. A standard Linear layer computes step 3 from step 3 only. With a weight of 2 it returns $2 \cdot 40 = 80$, and the values at steps 0–2 never enter the formula. TimeLinear computes step 3 from *all* steps, with the weights in row 3 of W . Two choices of W show what this gives:

$$\underbrace{\begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{pmatrix}}_{W = \text{running sum}} \begin{pmatrix} 10 \\ 20 \\ 30 \\ 40 \end{pmatrix} = \begin{pmatrix} 10 \\ 30 \\ 60 \\ 100 \end{pmatrix}, \quad \underbrace{\begin{pmatrix} 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}}_{W = \text{lag 1}} \begin{pmatrix} 10 \\ 20 \\ 30 \\ 40 \end{pmatrix} = \begin{pmatrix} 0 \\ 10 \\ 20 \\ 30 \end{pmatrix}.$$

With the first matrix, step 3 becomes $10 + 20 + 30 + 40 = 100$: the total flow so far, i.e. the volume. With the second, every step receives the value of the previous step. So each output step is a mix of several input steps, and row t of

W is the recipe for step t : $W_{ts} = 0$ means that step s is ignored, and $W_{ts} \neq 0$ means that it contributes with this weight. The network learns these recipes. Weights below the diagonal look into the past, weights above it look into the future. The future is allowed here, because the whole breath is known in advance. Mathematically, TimeLinear is still an ordinary linear layer, applied along the time axis instead of the feature axis. It is the same as `nn.Linear(80, 80)` applied to the transposed tensor $[B, 128, 80]$. This one change matters a lot: it is the only layer in the model that moves information between time steps. Every other layer works on each step separately. With six blocks, the mixing is repeated on increasingly complex features.

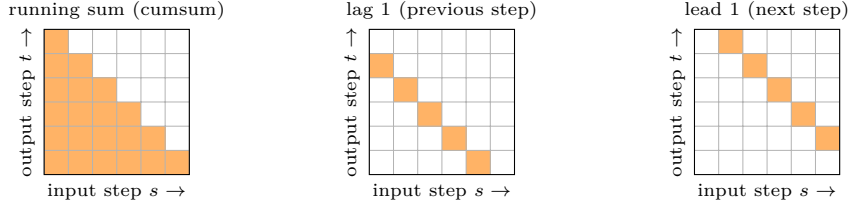


Figure 2: Hand-made features are special cases of W (shown for 6 steps; filled cell = 1, empty = 0). Lower-triangular ones give the running sum $\sum_{s \leq t} Z_s$, a sub-diagonal gives the lag, a super-diagonal gives the lead. The learned W is shown in Fig. 4.

This is why the layer suits the task (Fig. 2). Our best hand-made feature, the cumulative sum of u_{in} , is exactly TimeLinear with a lower-triangular matrix of ones; we checked this with an assert. Lags and leads are shifted identity matrices. Instead of fixing these patterns by hand, the network learns the matrix that fits the data. Because it is applied to 128 learned channels rather than to raw u_{in} , it can, for example, accumulate an estimated air flow to obtain the volume, the $V = \int Q dt$ term of $P = RQ + V/C$. In code it is an `nn.Module` with two `nn.Parameters` (W initialised Kaiming-uniform, b with zeros) and `forward(x) = self.weight @ x + self.bias[:, None]`. For a batch x of shape $[B, 80, 128]$ the matrix product is broadcast over the batch dimension automatically.

5.4 Residual block and the full model

Each block updates the hidden state in two steps: first the steps exchange information through TimeLinear, then every step is processed on its own by the MLP. Neither step replaces $\mathbf{H}$; each *adds a correction* to it:

$$\begin{aligned}\tilde{\mathbf{H}}^{(\ell)} &= \mathbf{H}^{(\ell-1)} + \text{Drop} \left(\text{ReLU} \left(\text{TimeLinear}_{\ell}(\text{LN}(\mathbf{H}^{(\ell-1)})) \right) \right) \\ \mathbf{H}^{(\ell)} &= \tilde{\mathbf{H}}^{(\ell)} + \text{MLP}_{\ell} \left(\text{LN}(\tilde{\mathbf{H}}^{(\ell)}) \right), \quad \text{MLP}(\mathbf{Z}) = \text{Drop} \left(\text{ReLU}(\mathbf{Z}A + \mathbf{1}a^{\top}) \right) B + \mathbf{1}c^{\top}\end{aligned}$$

where $A \in \mathbb{R}^{128 \times 256}$, $B \in \mathbb{R}^{256 \times 128}$ and the biases a, c are the weights of the two `nn.Linear` layers (shared by all 80 steps). The full model is an input projection, six blocks and an output head:

$$\mathbf{H}^{(0)} = \mathbf{X}W_{\text{in}} + \mathbf{1}b_{\text{in}}^{\top}, \quad \mathbf{H}^{(1)}, \dots, \mathbf{H}^{(6)} \text{ by the block above,} \quad \hat{\mathbf{y}} = \text{LN}(\mathbf{H}^{(6)})w_{\text{out}} + b_{\text{out}}\mathbf{1},$$

with $W_{\text{in}} \in \mathbb{R}^{31 \times 128}$ turning the 31 features of each step into 128 channels and $w_{\text{out}} \in \mathbb{R}^{128}$ turning them back into one pressure value per step.

Three design choices make the 12 stacked sublayers trainable. *Residual connections*: the block only learns a correction, and the derivative of the identity path is 1, so the gradient reaches the first layers without vanishing. *LayerNorm before each sublayer* keeps the scale of the activations stable from block to block, and unlike BatchNorm it does not depend on the batch. *Dropout* in both sublayers regularises the model. The R_C configuration enters as 9 one-hot columns of $\mathbf{X}$. Multiplying a one-hot vector by W_{in} simply selects one row of W_{in} , which is exactly what an embedding does, so we did not add a separate `nn.Embedding`.

5.5 Training

Loss. The competition scores only inspiration, so the loss is the MAE over scored steps: $\mathcal{L} = \sum_t M_t |\hat{y}_t - y_t| / \sum_t M_t$, summed over all breaths of the batch (`11_loss(pred[mask], target[mask])`). Expiratory steps get no direct gradient, but they still serve as context: TimeLinear lets them influence the scored steps. An auxiliary Δp head and **EarlyStopping** were planned and cut as unnecessary.

Custom optimiser. We implemented AdamW from scratch as a subclass of `torch.optim.Optimizer`. For every parameter θ with gradient g at update number t it keeps two running averages: m , the average gradient (the direction to move, “momentum”), and v , the average squared gradient (how large the gradients of this parameter usually are). One step is

$$\theta \leftarrow \theta(1 - \eta\lambda), \quad m \leftarrow \beta_1 m + (1 - \beta_1)g, \quad v \leftarrow \beta_2 v + (1 - \beta_2)g^2, \quad \theta \leftarrow \theta - \eta \frac{m/(1 - \beta_1^t)}{\sqrt{v/(1 - \beta_2^t)} + \varepsilon}.$$

Here η is the learning rate, λ the weight decay, $\beta_1 = 0.9$ and $\beta_2 = 0.999$ control how long the averages remember, the factors $1 - \beta^t$ correct the bias of averages that start at zero, and $\varepsilon = 10^{-8}$ avoids division by zero. Dividing by $\sqrt{v}$ gives

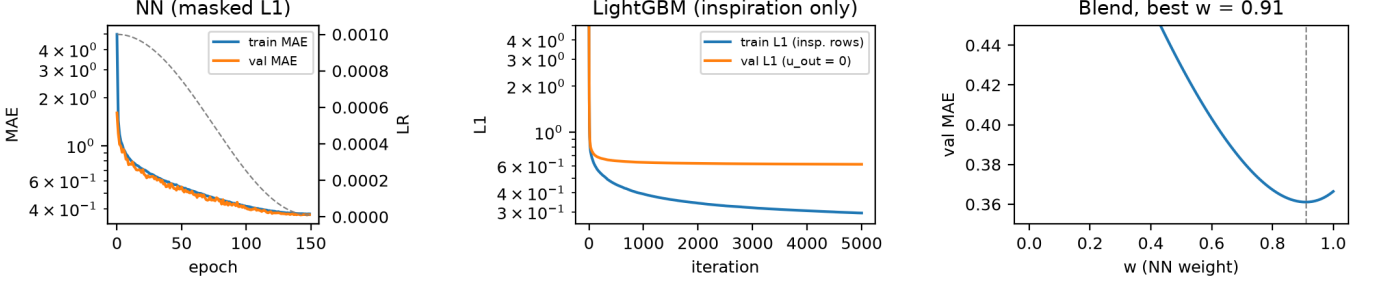


Figure 3: Optimisation curves. Left: NN train/val MAE per epoch and the cosine LR. Middle: LightGBM L1 per tree. Right: validation MAE of the blend $w \cdot \text{NN} + (1 - w) \cdot \text{LGBM}$, minimum at $w = 0.91$.

Model	Val MAE	Val MAE (snap)	RMSE	ME	R^2	Kaggle public / private (snap)
Median constant / linear regression	7.516 / 4.97	—	—	—	— / 0.49	—
LightGBM, all rows	0.6199	0.6194	0.990	+0.015	0.9884	—
LightGBM, inspiration only	0.6157	0.6153	0.984	+0.012	0.9886	0.6138 / 0.6160
NN (TimeLinear)	0.3664	0.3658	0.590	+0.002	0.9959	0.3698 / 0.3718
Mean of NN and LGBM	0.4294	0.4288	0.675	+0.007	0.9946	—
Blend 0.91 NN+0.09 LGBM	0.3611	0.3605	0.581	+0.003	0.9960	0.3638 / 0.3659

Table 2: Metrics on $u_{\text{out}} = 0$ of the validation breaths. Without snapping Kaggle is ≈ 0.0006 worse (blend 0.3644/0.3665).

every parameter its own step size. The first update shrinks the weights directly; this is the “W” in AdamW (decoupled weight decay), so the shrinkage is not distorted by the adaptive scaling. Weight decay $\lambda = 10^{-2}$ is applied only to weight matrices; biases and LayerNorm parameters use $\lambda = 0$. One pitfall: Lightning passes the forward and backward pass to `step(closure)`, so `step` must call `closure()` under `torch.enable_grad()` first; otherwise the model silently does not train. After 10 steps our optimiser matched `torch.optim.AdamW (allclose)`.

width d / MLP	128 / 256	optimiser	custom AdamW, $\eta_0 = 10^{-3}$, $\beta = (0.9, 0.999)$, $\lambda = 10^{-2}$
blocks	6 (441,953 params)	LR schedule	cosine, updated every step (35,400), $10^{-3} \rightarrow 10^{-5}$
dropout	0.1	regularisation	dropout, decoupled WD, LN, grad-clip $\ g\ \leq 1$
batch	256 breaths	training	150 epochs, best <code>val_mae</code> checkpoint (epoch 147), 43 min

Table 1: NN hyperparameters (Lightning LitPressure; MLflow logs hparams, curves, LR and per-layer gradient norms).

6 Results

Figure 3 shows the optimisation curves and Table 2 compares all models on the same validation breaths. Besides the two single models we blend them, $\hat{y} = w \hat{y}_{\text{NN}} + (1 - w) \hat{y}_{\text{LGBM}}$, choosing w on a grid of 101 values. The blend weight is a single scalar fitted on 458k validation rows, so it cannot overfit, and its Kaggle score confirms it. NN and LGBM errors correlate at only 0.44: the models partly make different mistakes, so a small LGBM weight cancels some NN errors, while the plain mean is worse than the NN alone because LGBM is much weaker.

7 Model insights and conclusions

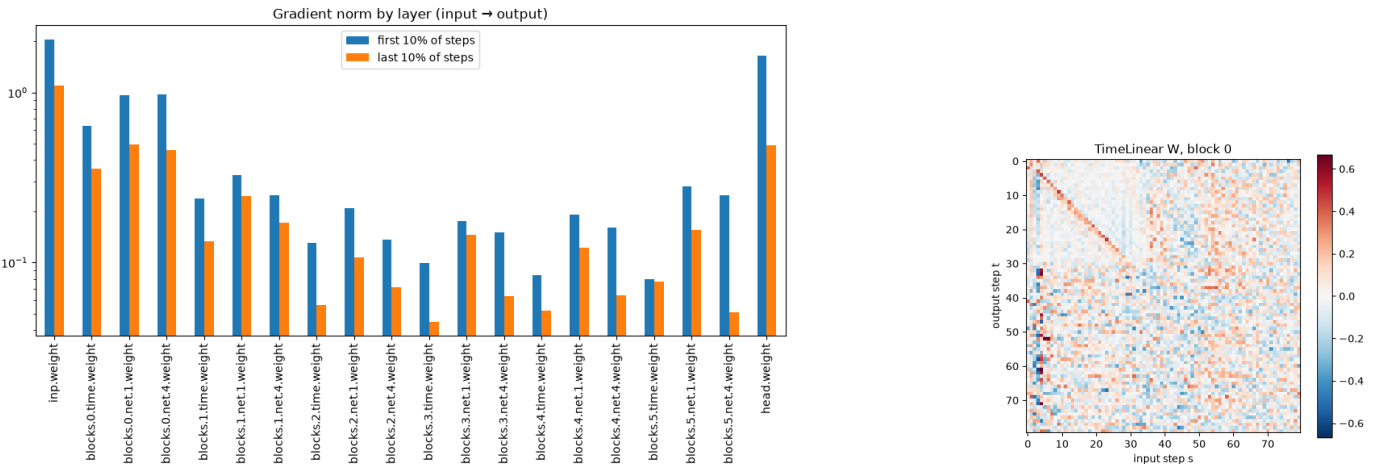


Figure 4: Left: L_2 gradient norm per weight matrix (input $\rightarrow$ output), first vs. last 10% of steps. Right: learned W of block 0.

The validation can be trusted. For every model, val MAE is within 0.001–0.006 of the Kaggle score, and ME is close to zero. The NN is slightly better on val than on Kaggle, because its checkpoint was chosen on val.

Seeing the whole breath matters most. With the same 31 features, the NN is 40% better than LightGBM (0.366 vs 0.616). LightGBM overfits: its train MAE is half of its val MAE (Fig. 3). The NN does not overfit: its train and val MAE are almost equal (0.373 and 0.368) and still go down at the end of training.

TimeLinear learned a meaningful matrix (Fig. 4, right). For the inspiration steps ($t \lesssim 30$), W is almost lower-triangular with a strong diagonal. So each step adds up the past steps, which is how volume $V = \int Q dt$ is formed. The rows for expiration ($t \gtrsim 32$) stay close to random: the loss ignores these steps, so nothing trains them.

The gradients flow well. They neither vanish nor explode, and all layers stay within two orders of magnitude. The middle blocks get the smallest gradients, so 4 blocks would probably be enough.

Snapping predictions to the 950-level sensor grid always helps a little, by about 0.0005 MAE.